

Translating Mizar for First Order Theorem Provers

Josef Urban

Dept. of Theoretical Computer Science
Charles University
Malostranske nam. 25, Praha, Czech Republic
`urban@kti.ms.mff.cuni.cz`

Abstract. The constructor system of the Mizar proof checking system is explained here on examples from Mizar articles, and its translation to untyped first-order syntax is described and discussed. This makes the currently largest library of formalized mathematics available to first-order theorem provers.¹

1 Introduction, Previous Work

Mizar [Rudnický 92] is a system for computer checked mathematics. In more detail, Mizar is associated with several things:

- The Mizar language ... this is the language in which Mizar articles must be written, so that they can be checked by computer.
- The Mizar Mathematical Library (MML). This is the growing (now about 700) collection of Mizar articles that have already been written and computer checked and the notation, definitions, theorems and other Mizar constructs created in them can be used for writing new Mizar articles. Various presentations of the MML exist today: Formalized Mathematics, online html-ized abstracts, Mizar Encyclopedia.
- The Mizar checker and other software utilities for working with articles and MML.
- The Mizar project headed by A.Trybulec, taking care of the things named above as well as other things related to Mizar.

Several introductions to the Mizar language as well as to the practical aspects of writing Mizar articles exist today e.g. [Bonarska 90, Muzalewski 93, Rudnický 92] or [Wiedijk 99]. It is recommended to have a look at one of them, but we will try here to explain the features of Mizar, relevant for the first order translation.

The largest previous attempt at first order translation of the MML was done by the ILF group [Dahn 97]. However, the work remained unfinished and only

¹ A more detailed version of this article will be a part of author's PhD thesis.

several basic articles were translated. The translation we present is done uniformly for the whole first order part of MML², and tested even for very advanced MML articles.

2 General Descriptions

2.1 The Mizar Language

The detailed specification of the Mizar language is given in [Syntax] and notions defined there will be written in *italics* here. We recommend to have a look at a sample Mizar article (e.g. article about real numbers [REAL_1]), to see examples of the abstract syntax. The logic (especially constructor and type system) of Mizar has not been formalized yet (strictly speaking, it is formalized in the Mizar sources). So our approach is to follow the [Syntax], explaining what the language constructs mean in the Mizar implementation, and map them into untyped first order syntax. The most practical “proof” of the correspondence of the translation with the Mizar sources, is then the correspondence of the results of the Mizar checker with the results of theorem provers run on translated tasks.

The main parts of a Mizar article are definitions (*Definitional-Items*, (*Functor*, *Predicate*, *Mode*, *Structure and Attribute-Definition*)), *Theorems* and *Schemes*. Attributes are predicates handled specially by the system, according to the rules defined by the user in *Cluster-Registrations*. All Mizar terms are typed. There is a largest (default) type called “**set**” or “**Any**”. All other types have one or more mother types. Types of variables are given either in global *Reservations* or local *Loci-Declarations* or inside quantified formulas. Types of other terms are computed according to *Functor-Definitions*. Types can have arguments (be parameterized) in Mizar, e.g. “**Element of X**” or “**Function of NAT, REAL**” are legal types with one or two arguments, respectively. Type translation is the largest part of the first order translation. There are two possible basic approaches to type translation³(we use the second approach, which is better for SPASS).

- Types can be thought of as set-theoretic classes (e.g. type “**set**” being the universal class, type “**Element of X**” being the set of all elements of X, type “**Integer**” being the set of all integers, etc.).
- Types can be thought of as predicates thus “**set(X)**” is true for any X, “**Integer(X)**” is true iff X is integer, and “**Element(X,Y)**” is true iff $X \in Y$

2.2 Syntactic Levels

The Mizar language handles a very large database of articles about different parts of mathematics and this necessarily leads to notation conflicts. The solution to such conflicts is introduction of two syntactic levels of the language: the **pattern** level and the **constructor** level.

² For the precise meaning of this, see section Unimplemented Features of Mizar.

³ [Dahn 98] suggests another, based on inclusion operations ([Goguen 92]), but the system of attributes (and thus vast multisortedness of MML terms) makes it probably less efficient than the standard translations.

Constructors are the real unambiguous functions, predicates, types and attributes to which the patterns are translated before any proof checking takes place. Patterns are mapped to the constructors; they accommodate the need for having different symbols for the same constructor or vice versa (same symbol for different constructors). The process of mapping patterns to constructors is done separately for each Mizar article depending on its *Environment-Declarations* and is usually quite nontrivial.

Example 1. binary symbols “in” and “<” define different patterns, but when dealing with ordinals, they are mapped to the same constructor, i.e. “A in B” and “A < B” cannot be distinguished when translated to the constructor level.

Definitions can influence both the pattern and the constructor level. Typically, a definition causes a new pattern and also a new constructor to be defined, but in many cases there is no effect on the constructor level. For the purpose of translating Mizar articles into first order syntax, only the constructor level is important, the patterns can be thought of as a “syntactic sugar” added on top of the constructors.

2.3 First Order Formats, Outline of the Translation

Several kinds of first order syntax are used today for first-order provers, e.g. TPTP format [Suttner and Sutcliffe 98], DFG format [Hahnle et al 96], Otter format [McCune 94] and others. We chose direct translation into DFG format, since our immediate purpose was to experiment with the SPASS prover [Weidenbach et al 99]. More generic approach is certainly desirable, however, at a first glance, several such approaches come into mind (e.g. ILF-like approach [Dahn 97], TPTP-like approach, or even adding direct Mizar support for other formats than just DFG), so we postponed such decisions for later time. Note that the `dfg2tptp` tool can be used for translation to TPTP and thus (using TPTP tools) to virtually any other syntax.

So the general approach to translation is following:

- We use parts of Mizar (Parser, Analyzer) which translate the Mizar articles to the constructor level, where our first order translation starts.
- We give absolute (context independent) names to all constructors⁴.
- Definitions of constructors usually translate to several first order formulas, since we have to translate both the type hierarchy information given in the definitions and the actual *Definiens*.
- Sometimes also additional properties (e.g. commutativity, transitivity, etc.) of the defined constructors are stated inside definitions. They are also translated into corresponding first order formulas.
- All formulas are relativized with respect to the typed variables occurring in them (using the above mentioned predicate translation of types). So e.g. universally quantified Mizar formula

⁴ The naming scheme we use in what follows has obviously no importance for theorem provers, however, it has become standard in various Mizar presentations, allowing their interoperability.

“for x being Real holds $x-x = 0$ ” translates to first order formula
 “ $\text{Real}(x)$ implies $x-x = 0$ ”.

Remark: The actual DFG syntax translation of the above mentioned Mizar formula is:

```
forall([B1], implies(v1_arytm(B1), equal(k3_real.1(B1,B1),0))).
```

However, for explanation purposes, we use here rather the user-defined symbols instead of the absolute constructor names, and also for the sake of better readability, we do not strictly adhere to the DFG syntax of formulas.

Next we explain in more detail the translation of various Mizar definitions.

3 Translation of Mizar Definitions

3.1 Mode Definitions

Types in Mizar can be defined using either *Mode-Definitions* or *Structure-Definitions*. We deal with modes first. Since structures are more complicated, we postpone them after the explanation of functions.

As already stated, the translation of the largest (default) type “**set**” is a predicate that always holds true (“**set**(X)” is true for all terms X). So omitting relativization by this predicate is logically correct, and we do it for the sake of better readability of translated formulas. For all other modes, the syntactic structure of *Mode-Definitions* is:

```
mode Mode-Pattern ( [ Specification ] [ means Definiens ]
Correctness-Conditions ; | is Type-Expression ; ) { Mode-Synonym } .
```

Example 2.

```
definition let X;
  mode Element of X means
:Def2: it in X if X is non empty
  otherwise it is empty;
  existence by B00LE:def 1;
  consistency;
end;
```

This is a definition of the mode “**Element of X**” from the article `subset.1.miz`. The *Loci-Declaration* “**let X;**” declares the variables occurring in the definition. This definition omits the optional *Specification* part. If the *Specification* is present in the definition, it gives a mother type for the newly defined mode. If not, the largest type “**set**” is used as the mother type.

The *Definiens* after the keyword “**means**” consists of an optional *Label-Identifier* and either a simple sentence or several sentences separated by the keywords “**if**” and “**otherwise**”. The latter (*Conditional-Definiens*) is used for “per cases” definitions.

In our example, the *Conditional-Definiens* distinguishes the case when the mode argument X is non empty from the case when the mode argument X is empty. The sentence for the first case states, that for any “**it**” (special variable

used in definitions) being of the type “**Element of X**” means “it in X”. The second case sentence says, that in the degenerate case when X is empty, being of the type “**Element of X**” means to be empty too⁵.

The keywords “**existence**” and “**consistency**” in the example introduce *Correctness-Conditions*. The “**existence**” condition states that the extension of the defined type is non empty. The “**consistency**” condition states that the disjunction of all the cases into which the definition was split, is true. The absolute name given to the mode constructor created in this definition is “**m1_subset_1**”, which means that it is the first mode constructor in article **subset_1**,

We translate types as predicates, so n-ary types will become n+1-ary predicates, and e.g. the Mizar type qualification “X is **Element of Y**” will be translated as “**m1_subset_1(X,Y)**”.

Next we want to encode the part of the type hierarchy created by this definition. As noted above, the mother type of “**Element of X**” is the largest type “**set**”. The translation would be following:

```
‘forall([X,Y], m1_subset_1(Y,X) implies set(Y)).’
```

However, as already mentioned, “**set(Y)**” is always true and we chose not to translate it, so in such cases (i.e. when the mother type is “**set**”), we do not create the type hierarchy translation.

The definiens translates to the following formula:

```
‘m1_subset_1(Y,X) iff ( ( not(empty(X)) and in(Y,X) )
or ( empty(X) and empty(Y) ) )’
```

(instead of “**empty**” and “**in**” their absolute names would be used).

Finally, we translate the existence condition:

```
‘forall([X], exists([Y], m1_subset_1(Y,X))).’
```

3.2 Predicate and Attribute Definitions

In comparison to first order predicates, Mizar predicates can have several predefined properties (e.g. reflexivity, symmetry, etc.), and they also define the type of their arguments. *Predicate-Definition* has the following syntactic structure:

```
pred Predicate-Pattern [ means Definiens ] ; Correctness-Conditions
{ Predicate-Property } { Predicate-Synonym } .
```

e.g.:

```
definition let X,Y;
  pred X c= Y means
:: TARSKI: def 3
  x in X implies x in Y;
  reflexivity;
end;
```

This is a definition of the subset relation from article **tarski**. Again first comes a *Loci-Declaration* “**let X,Y;**”, after that the *Predicate-Pattern* “**X c=**”

⁵ If the reader is curious about why the degenerate case is handled this way, the answer is, that simply in many cases it turned out to be advantageous.

Y”, then the *Definiens* “ x in X implies x in Y ,” and finally one *Predicate-Property* is stated: “**reflexivity**,”. *Correctness-Conditions* occur only in redefinitions of predicates, redefinitions will be discussed later.

Again, this definition creates both a pattern and a constructor, the standard symbol for predicate constructors is “**r**”, so the absolute name would be “**r1.tarski**” here.

The *Definiens* formula is simply translated as equivalence:

```
‘‘r1.tarski(X,Y) iff (x in X implies x in Y)’’
```

Predicate-Property can be:

```
symmetry connectedness reflexivity irreflexivity
```

So we add the corresponding theorem for such properties, here it is:

```
‘‘r1.tarski(X,X).’’
```

Attributes are very similar to predicates, so due to space constraints, their explanation is omitted here.

The *Cluster-Registrations*, which define rules of attribute handling will be explained later.

A possibly negated attribute is called *Adjective* in Mizar. A finite set of *Adjectives* is called *Adjective-Cluster* (or just cluster) in Mizar. Clusters can be used as prefixes to types, e.g. “finite non empty set” uses the *Adjectives* “finite” and “non empty” as a prefix to the type “set”. Such types with non empty *Adjective-Cluster* are translated as a conjunction of all the predicates corresponding to the attributes and the predicate corresponding to the underlying type, so here it is:

```
“finite(X) and not(empty(X)) and set(X)” for a variable X of the type
“finite non empty set”.
```

Again, the “set” can be pruned, so the translations is just “finite(X) and not(empty(X))”.

3.3 Functor Definitions

Functions in Mizar have to define the types of their arguments and the type of their result. Several *Functor-Properties* (e.g. “commutativity”) can be associated with them.

The syntax of *Functor-Definition* is:

```
func Functor-Pattern [ Specification ] [ ( means | equals ) Definiens ]
; Correctness-Conditions { Functor-Property } { Functor-Synonym } .
```

Example 3.

```
definition let x be real number;
  func -x -> real number means :Def1: x + it = 0;
  existence by AXIOMS:19;
  uniqueness by Th10;
```

The initial *Loci-Declaration* declares a variable x for the type “number” with *Adjective* “real”. Type “number” is now defined just as a convenient synonym for the largest type “set” (in article *arytm*), “real” is an attribute defined also in article *arytm*. The *Functor-Pattern* is “ $-x$ ” here, and the *Specification* “ $->$

real number” defines the result type of the function. The *Definiens* “:Def1: $x + it = 0$;” is the definitional formula of the defined function.

The *Correctness-Condition* “**existence**” states, that there exists an object (of the desired type “**real number**”) conforming to the *Definiens*. “**uniqueness**” says that such object is unique (proved by previous theorem Th10). Before accepting a new *Functor-Definition*, the system always checks these two conditions.

This definition creates one new pattern and one new constructor. The standard Mizar symbol for functor constructors is “**k**” so the constructor’s absolute name is “**k1_real_1**” (first functor in article **real_1**). We need to translate the typing, the *Definiens* and possibly the *Functor-Properties* (none in this case).

The typing translates to:

“‘**real number**’(X) implies ‘**real number**’(k1_real_1(X)).”

Following the note above about *Adjective-Clusters*, ‘**real number**’(X) translates to “(real(X) and number(X))” and since “**number**” is just a synonym for “**set**” it is pruned to just “real(X)”. Since “**real**”’s absolute name is “v1_arytm”, the exact translation is:

“forall([X], implies(v1_arytm(X), v1_arytm(k1_real_1(X))))”.

The *Definiens* is translated by first instantiating the special variable “it” with the *Functor-Pattern*, yielding:

“ $x + (-x) = 0$;” and translating the result formula, which gives:

“equal(+ (x, k1_real_1(x)), 0)”.

After relativization and replacing “+” with its absolute name “k3_arytm”, we get:

“forall([X], implies(v1_arytm(X), equal(k3_arytm(X, k1_real_1(X)), 0)))”

3.4 Structure Definitions

Structures correspond (to some extent) to the “product” types of various other languages. The syntax of *Structure-Definition* is:

struct [(*Ancestors*)] *Structure-Symbol* [*over Loci*] (# *Fields* #) ;

Example 4.

```
struct(1-sorted) TopStruct (# carrier -> set,
                           topology -> Subset-Family of the carrier #);
```

This is definition of the structure TopStruct from the article about Topological Spaces “pre_topc”⁶.

Structures can define their ancestor structures. All *Fields* of an ancestor must also be *Fields* of the defined structure. In our example, there is given one ancestor “1-sorted”. *Ancestors* of structures are treated in a way similar to the treatment of mother types of modes by the system, i.e. they also define the type hierarchy.

The *Structure-Symbol* is “TopStruct” in our example. The optional syntax “[*over Loci*]” is not used there. If used, it parameterizes the structure in the

⁶ This is not yet a topological space, just the underlying structure.

same way as modes can be parameterized, i.e. structures can also have any number of arguments. Finally, the structure defines two *Fields*:

“carrier” of the type “set”, and

“topology” of the type “Subset-Family of the carrier”.

We will now explain the effects of the *Structure-Definition* on the constructor level. Any structure defines a new type constructor. Such constructors are also called “aggregate types”, and the standard Mizar symbol for them is “1”. So the absolute name of the aggregate type in our example is “l1_pre_topc”. Such types behave in exactly the same way as the mode types, i.e. variables can be reserved for them, *Adjective-Clusters* can be added to them as a prefix, etc. *Ancestors* become the mother types of the new aggregate type, so in our example, there is one mother type “1-sorted” (its absolute name is “l1_struct_0”).

The *Fields* of a structure give rise to the so called “selector constructors”. The standard Mizar symbol for selectors is “u”. In our case, the structure contains two selectors, but the selector “carrier” is inherited from the ancestor “1-sorted” (l1_struct_0), where it was defined for the first time. So its absolute name is “u1_struct_0”. The selector “topology” is defined for the first time in our example, so its absolute name is “u1_pre_topc”.

The meaning of a selector is a function operating on the aggregate type. So e.g. “carrier” takes arguments of the type “1-sorted” (or any of its specializations, like TopStruct) and its result type is “set”.

Finally, we need something to create the desired structure, when the *Fields* are given. This is solved in Mizar by creating “aggregate functor” for the structure. The standard Mizar symbol for aggregate functors is “g”, so for TopStruct, its absolute name is “g1_pre_topc”. The symbol for the aggregate functor is in Mizar the same as the symbol for the aggregate type, i.e. TopStruct. Given properly typed arguments, e.g. by following global reservations:

“reserve X for set;”

“reserve Y for Subset-Family of X;”

the Mizar term “TopStruct(#X,Y#)” means applying the aggregate functor “TopStruct” (“g1_pre_topc”) to arguments X and Y, yielding an object of the aggregate type “TopStruct” (“l1_pre_topc”).

Obviously, it holds then that

“X = the carrier of TopStruct(#X,Y#)” and

“Y = the topology of TopStruct(#X,Y#)”.

Aggregate types are translated again as predicates, selectors and aggregate functors are translated as functions.

Axiom stating mutual interdependence of these constructors has to be added, this is done by stating that the structures are freely generated by its *Fields*, using the aggregate functor:

“TopStruct(X) implies X = TopStruct(carrier(X),topology(X));”,

in absolute notation:

“forall([X], implies(l1_pre_topc(X),

equal(X,g1_pre_topc(u1_struct_0(X),u1_pre_topc(X))))”).

3.5 Cluster Registrations

We have already mentioned, that *Cluster-Registrations* define for the system the special rules of attribute handling. They either state non emptiness of attributed types (*Existential-Registration*) or define attribute propagation (*Conditional-Registration*, *Functorial-Registration*).

Existential-Registration. The syntax of *Existential-Registration* is:
`cluster Adjective-Cluster Type-Expression ; Correctness-Conditions.`

Existential-Registrations assert the existence (non emptiness of the extension) of a type with some *Adjective-Cluster* prefix.

Example 5.

definition

```
cluster cardinal set;
  existence
  proof ...
  end;
```

end;

in article `card_1` asserts the existence of the type “`set`” with the *Adjective* “`cardinal`”. Whenever a variable is being reserved for a type with some non empty *Adjective-Cluster*, the non emptiness of the type’s extension is checked by the system. We already noted how types with *Adjective-Cluster* are translated, so the translation is a simple existence formula in such cases:

“`exists([X], (cardinal(X) and set(X)))`”.

In absolute notation and pruning the “`set`”:

“`exists([X], v1_card_1(X))`”.

Conditional-Registration. The syntax of *Conditional-Registration* is:
`cluster Adjective-Cluster -> Adjective-Cluster Type-Expression ;`
Correctness-Conditions .

Conditional-Registrations give rules for attribute propagation, e.g.:

Example 6.

definition

```
cluster cardinal -> Ordinal-like set;
  coherence
  proof ...
  end;
```

end;

from article `card_1` states that to any “`set`” with *Adjective-Cluster* “`cardinal`” the system should add the *Adjective-Cluster* “`Ordinal-like`”. A proof of this must be given after the keyword “`coherence`”. *Conditional-Registrations* are easily translated as implication:

“set(X) and cardinal(X) implies Ordinal-like(X)”.

In absolute syntax and after “set” pruning:

“forall([X], implies(v1_card_1(X), v3_ordinal1(X)))”.

Functorial-Registration. The syntax of *Functorial-Registration* is:
 cluster *Term-Expression* -> *Adjective-Cluster* ; *Correctness-Conditions*

Example 7.

```
definition let X be finite set;
  cluster Card X -> finite;
  coherence;
end;
```

from article `card_1` says that for any X with the type “finite set”, the attribute “finite” applies also to the term “Card X” (the cardinality of X). While *Conditional-Registrations* define attribute propagation for some underlying type (“set” in the previous example), *Functorial-Registrations* do this for terms (“Card X” here). Their translation is also simple:

“(finite(X) and set(x)) implies finite(Card(X))”.

In absolute syntax and after “set” pruning :

“(forall([X], implies(v1_finset_1(X), v1_finset_1(k1_card_1(X))))))”.

4 Redefinitions

In Mizar, there can be predicate, attribute, type and functor redefinitions. Apart from the keyword “**redefine**”, they have the same syntactic structure as the respective definitions. They can introduce new notation, change the definitional formula or the result types, or add new properties. Some redefinitions do not touch the constructor level, they only define new notation.

If the constructor level is touched by the redefinition, Mizar usually creates a new constructor, but knows to some extent, that it is equivalent to its redefined counterpart. The treatment in Mizar is not complete, e.g. if several type redefinitions are possible, only the last defined is chosen, since Mizar implementation sometimes requires the terms to have unique result types. This is no serious restriction for Mizar, since Mizar allows explicit typing (using the “is” keyword) of terms.

Since we translate types (and attributes) as predicates, there is no trouble in having multiple “types” for a term. After the translation, these are simply several unit clauses holding about the term, e.g.:

“finite(A) and Ordinal(A) and Integer(A) and Real(A)”.

So unlike Mizar, we do not create new constructors for redefinitions, we just translate the new facts stated by the redefinitions.

5 Unimplemented Features of Mizar

Mizar *Schemes* and Fraenkel terms are not currently translated, numbers are translated only in a very simple way.

Mizar *Schemes* are used to express second-order assertions parameterized by functor or predicate variables. This makes it possible, to state principles like Separation or Induction in Mizar, e.g.:

```
Example 8. scheme Separation { A()-> set, P[set] } :
  ex X st for x holds x in X iff x in A() & P[x]
  proof ...
  end;
```

from article `boole` states that for any set “A()” and any formula “P” with a free variable of the type “set”, there is a subset of “A()” of elements with the property “P”.

Another feature of Mizar is the Fraenkel (“setof”) operator, which (in accordance with the Separation principle) creates sets of elements that satisfy a given formula.

```
Example 9. {i where i is Nat: i < 5}
```

is the set of natural numbers smaller than 5. The result type of such terms is the largest type “set”.

Schemes and Fraenkel terms cannot be directly translated into first-order syntax. These features are relatively rare in MML and their usage is quite restricted (full instantiation of a scheme must be given before it can be applied), so a very large part of MML can be translated even if these features stay unhandled. We think the performance gap between current first order provers and higher order provers justifies this omission.

Theoretically, the Fraenkel terms and *Schemes* could be completely removed using the standard set-theoretic elimination procedures (previous work with Otter in this direction is described in [Belinfante 96]), however, given the advanced constructor system in Mizar, this could lead to very large (and thus practically unusable) expansions. We believe, that the solution lies rather in modifying the current efficient first order provers to handle the “real mathematics” (at least to the extent the Mizar checker handles it today). This also applies to the Mizar built-in handling of numbers. The current (most simple) translation creates for each mentioned number a new constant. However, Mizar can evaluate some simple arithmetic expressions like “(5+3)*4”.

6 Related and Future Work

An important part of the translation are the scripts that generate proving problems from the translated Mizar database. Such scripts have to mimic the Mizar theory inclusion mechanism closely, add special axioms for things that are obvious to the Mizar checker, and possibly try to apply some filtering to handle large theories. Due to space constraints, we do not describe them here, they will probably be described in another article about experiments with theorem provers on the translated database.

Experimenting with theorem provers (currently E and SPASS) is work in progress at the time of writing this article. The main purpose behind the translation is to train theorem provers on the large database, and possibly also implement discovery systems that could make use of it.

The most recent experiments show, that the “well-typing” inferences that have to be carried out by the provers are much less efficient than the type system implementation in Mizar. So another work which is currently under way, is to add an efficient Mizar-like type system to theorem provers. It seems that implementation of type discipline will be a necessary feature for theorem provers to be able to handle large theories. For example, the SPASS prover, which implements Mizar types to some extent, seems to perform much better in advanced theories, than the heavily optimized recent CASC winners.

As already noted, another feature, that will eventually have to be added to theorem provers to handle theories based on ZFC (i.e. most of current mathematics), is efficient implementation of infinite schemes of first-order axioms. We also believe (cf. [Schumann 2001]), that provers should handle numbers efficiently.

We think a good way to boost the development of all these features for current theorem provers, would be to have e.g. a special part of CASC devoted to proving in large structured theory ensembles like MML. Detailed rules of such competition concerning e.g. allowed machine learning or type translation methods would have to be thought up.

References

- [Belinfante 96] Johan Belinfante, On a Modification of Godel’s Algorithm for Class Formation, Association for Automated Reasoning Newsletter, No. 34, pp. 1015 (1996)]
- [Bonarska 90] Bonarska, E., An Introduction to PC Mizar, Fondation Ph. le Hodey, Brussels, 1990.
- [Dahn 97] First Order Proof Problems Extracted from an Article in the MIZAR Mathematical Library. Ingo Dahn and Christoph Wernhard Proceedings of the International Workshop on First order Theorem Proving. RISC-Linz Report Series, No. 97–50, Johannes Kepler Universität Linz, 1997.
- [Dahn 98] Ingo Dahn. Interpretation of a Mizar-like Logic in First Order Logic. Proceedings of FTP 1998. pp. 137–151.
- [Goguen 92] J.Goguen and J. Meseguer. Order-sorted algebra I: Equational deduction formultiple inheritance, overloading, exceptions and partial operations. Theoretical Computer Science, 105(2):217–273,1992.
- [Hahnle et al 96] R. Hahnle, M. Kerber, and C. Weidenbach. Common Syntax of the DFGSchwerpunktprogramm Deduction. Technical Report TR 10/96, Fakultät für Informatik, Universät Karlsruhe, Karlsruhe, Germany, 1996.
- [McCune 94] McCune, W. W., OTTER 3.0 Reference Manual and Guide, Technical Report ANL-94/6, Argonne National Laboratory, Argonne, Illinois (1994).
- [Muzalewski 93] Muzalewski, M., An Outline of PC Mizar, Fondation Philippe le Hodey, Brussels, 1993.
- [REAL.1] Krzysztof Hryniewiecki, Basic properties of real numbers. Journal of Formalized Mathematics, 1, 1989.

- [Rudnicki 92] Rudnicki, P., An Overview of the Mizar Project, Proceedings of the 1992 Workshop on Types for Proofs and Programs, Chalmers University of Technology, Bastad, 1992.
- [Schumann 2001] J. M. Schumann, Automated Theorem-Proving in Software Engineering. Springer-Verlag, 2001.
- [Suttner and Sutcliffe 98] C. Suttner and G. Sutcliffe. The TPTP problem library (TPTP v2.2.0). Technical Report 9704, Department of Computer Science, James Cook University, Townsville, Australia, 1998.
- [Syntax] Syntax of the Mizar Language available online at <http://mizar.uwb.edu.pl/language/syntax.html>
- [Weidenbach et al 99] Weidenbach C., Afshordel B., Brahm U., Cohrs C., Engel T., Keen R., Theobald C. and Topic D., System description: Spass version 1.0.0, in H. Ganzinger, ed., '16th International Conference on Automated Deduction, CADE-16', Vol. 1632 of LNAI, Springer, pp 314–318
- [Wiedijk 99] Freek Wiedijk. Mizar: An impression. Unpublished paper, 1999. <http://www.cs.kun.nl/~freek/mizar/mizarintro.ps.gz>.